1. Background

- Motivation

  Following the previous year in Android application development for robotics system, we have been using Android app as the controller for the robots. However, this approach works with the xbox gamepad that acts as the main controller for our robots, using the buttons and joystick to navigate the robots manually. While this method provides real-time control, it limits automation and precise waypoint navigation.

  The initial idea of this project originated from Mr.PHOLL Chanphayuk, who proposed an Android-based interface that allows users to interact with the robot through touch inputs on the screen, combined with odometry feedback and a virtual joystick. The concept was intended for interactive tasks, such as robot basketball playing.

  After changes in term members and project direction, the focus of the project evolved from manual joystick control to a waypoint-based navigation system. This shift was encouraged by Dr.SRANG Sarot, aiming to reduce dependency on gamepad controllers and to explore a more autonomous navigation approach using Android applications as the main interface.

- Problem Statement

  Manual control using gamepads or joysticks presents several limitations in robotic navigation systems:

  - Continuous use input is required, increasing operator workload
  - Precise positioning is difficult due to human reaction time and control sensitivity
  - The system lacks scalability for autonomous behaviors

  To address these issues, a waypoint navigation system is proposed, where uses can define target positions (waypoints) through an Android application, and the robot navigates to these positions using odometry and orientation data.

  However, implementing such a system introduces challenges, including:

  - Accurate coordinate transformation between Android interface and the robot's reference frame
  - Proper handling of robot orientation (yaw) in navigation calculations
  - Reliable communication between the Android application and the robot's ROS2-based control system
  - Managing waypoint editing, execution, and return behaviors without causing state conflicts

  These challenges motivated the need to redesign the calculation logic and communication protocol to ensure compatibility with ROS2 and improve navigation accuracy.

- Literature Review

  Waypoint-based navigation is a common approach in mobile robotics, especially in autonomous systems using frameworks such as ROS and ROS2. Previous works and standard robotics literature describe waypoint navigation as a method where a robot moves through a sequence of pre-defined target positions using localization and motion control algorithms.

  Odometry-based navigation is widely used for estimating a robot's position by integrating wheel encoder data over time. Although odometry is simple to implement, it is sensitive to

drift and orientation errors, particularly in yaw estimation. Many systems combine odometry with higher-level navigation logic to mitigate these issues.

Mobile interfaces for robot control, including Android-based applications, have also been explored as flexible and user-friendly solutions. Touch-based interfaces allow intuitive interaction, but require careful mapping between screen coordinates and real-world robot motion.

This project builds upon these established concepts, focusing on practical implementation rather than theoretical optimization.

- Concepts

  The waypoint navigation system in this project is based on the following key concepts:
  - Waypoint navigation: the robot is instructed to move toward pre-defined target positions instead of being continuously controlled by the user.
  - Odometry: the robot's position and orientation are estimated using incremental motion data.
  - Yaw orientation: the robot's heading angle is critical for accurate navigation and must be correctly handled in coordinate transformations.
  - Client-sever communication: the Android application acts as a client that sends waypoints data to the robot, while the robot processes these commands within a ROS2 environment.

  For simplification, the system assumes:
  - A planar (2D) environment
  - Reliable odometry data without sensor fusion
  - Pre-defined motion control handled by the robot's existing control stack

  The Android application focuses on target setting, waypoint management, and command transmission, while the robot handles low-level motion execution.

2. Objective

This project is developed in order to:

- Design and implement Android app to define, edit, and manage navigation waypoints on a pre-defined map
- Develop custom ROS2 control logic to receive waypoint data and execute navigation commands on the robot
- Implement accurate coordinate transformation and scaling between Android interface and the robot's reference frame
- handle robot orientation (yaw) correctly during waypoint navigation to improve positioning accuracy
- Establish reliable client-server communication between the Android app and the ROS2-based robot system
- Implement waypoint execution features including sequential navigation, editing, and return-to-previous-waypoint behaviors
- Test and evaluate system performance, identifying limitations related to map alignment, yaw accuracy, and state management.

3. Scope of works

This project covers:

- Android application development (Target_Setter)
- Communication system

- Robotic software development (ROS2)
- System integration and testing

This project does not include:

- Full autonomous robot navigation
- Sensor fusion beyond basic odometry and IMU integration
- Algorithm optimization
- Low-level communication protocol
- Hardware-level motor driver design
- Network security mechanism

4. Methodology

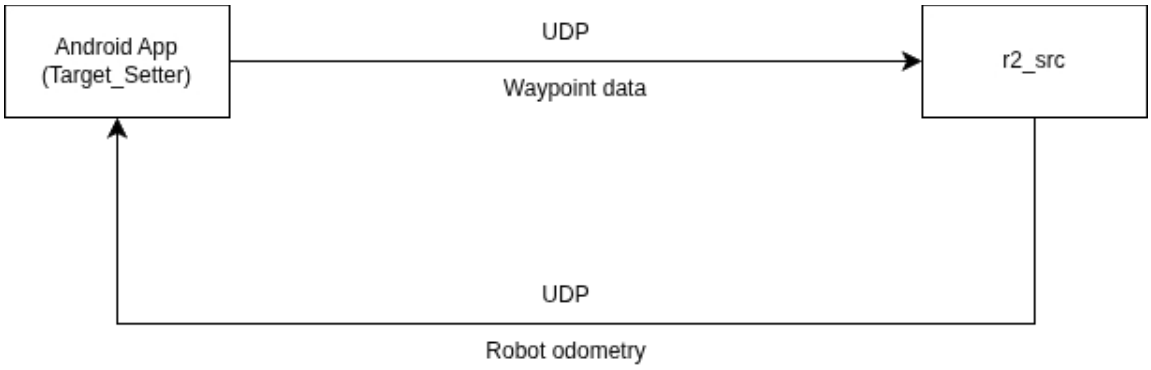
This section describes the development and implementation process of the waypoint navigation system using Android app: Target_Setter. The project consists of 2 main components: the Android application (Target_Setter) and the robot's source code (r2_src).

• System overview

The system is designed as a client-server architecture, where:

- The Android app acts as the client, allowing users to define, edit, and manage waypoints
- The robot acts as the server, receiving waypoints via UDP, executing navigation commands and sending odometry data back to the application

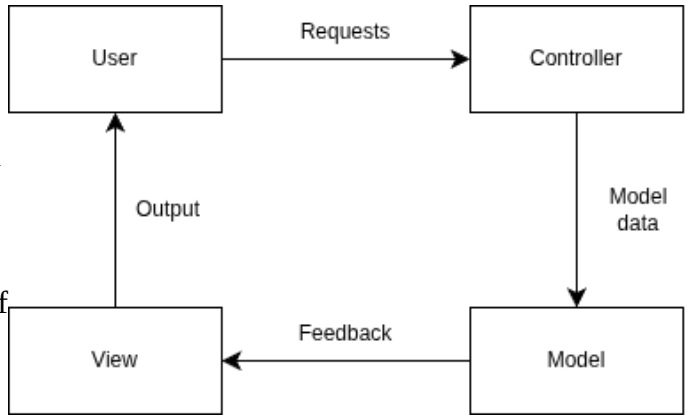
The workflow can be summarized as:



• Android application

- Architecture

  The app follows a Model-View-Controller (MVC) pattern. MVC architecture is a three-layered development architecture used in web-based applications and mobile app developments.



  - Model: this layer manages the information in the form of data that represents the output, consists of the application data, logic definition, function specification, business rule involvement. In the context of this

waypoint navigation system, the View layer of the app stores waypoints, odometry, and plan IDs.

- View: View layer allows the users to see the components of the app. View contains logic and markup required for presenting data to the users but not the complex logic that creates, stores, or manipulates the model. In Target_Setter, View displays the gamefields, robot position, and UI elements.
- Controller: Controller respond to the user's requests, acts as an interface between the users and the system, and perform necessary tasks on the data received from the users to carry out execution and alter the state of the app. This layer handles user inputs, waypoint managements, and communication with the robot.

We chose the MVC architecture because:

- The app needs asynchronous communication between itself and the robot, this choice eliminates the conflicts among the UI threads and network threads
- The app doesn't need to reload everything all at once. It renders UI elements only when it is requested from the users.
- The workflow and the data is easier to manage in this architecture.

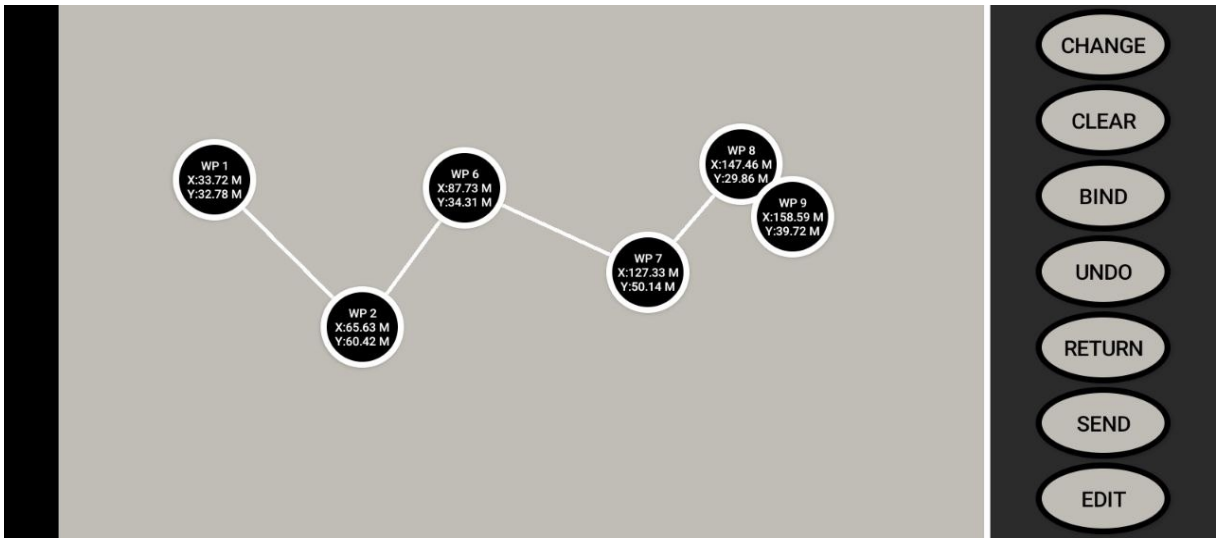
However, as the new functionalities are included in the app, this architecture becomes more complex to manage everything. This causes race conditions in asynchronous operation. A more suitable architecture would be MVVM (Model-View-ViewModel) architectural pattern.

- UI design

  The UI of Target_Setter is designed to provide an interface for setting, editing, and managing robot navigation waypoints. It allows the users to interact with the robot in a touch-bases environment, visualizing robot position and planning path in real-time.

  - Layout

    The main screen displays a top-down map of the gamefield. Users can tap on the map to add waypoints, which appear as circle buttons with text displaying the coordinate of the robot in the gamefield frame as the global frame. The left pane provides containers for the buttons.



The control panel contains buttons for:

- Clearing waypioints
- Udo-ing waypoint
- Editing waypoint
- Connecting the app to the robot
- Sending waypoints data
- Changing the gamefield parameters

This visual interface is built using ConstraintLayout. ConstraintLayout allows UI components to be positioned using relative and percentage-based constraints rather than fixed pixel values. This approach ensures that the UI scales correctly across different screen sizes and resolution.

Waypoint positions on the map are stored as percentage-based coordinates, which are later converted into pixel coordinates for rendering and meter-based coordinates to send to the robot. This design choice simplified coordinate scaling and maintains consistency betweem the Android interface and the robot's navigation frame.

- Interaction flow

  The interaction flow of Target_Setter is designed to guide the user from initial configuration to waypoint execution in a structured and predictable manner. Each interaction follows the MVC principle, where user actions are handled by the Controller, stored in the Model, and reflected in the View.

  - Field parameters inputs

    Before waypoint creation, the user configured the gamefield parameters, including Width and Height of the gamefield. These parameters define the reference frame used for coordinate transformation between the Android interface and robot's navigation system. Once set, the parameters are stored in the Model and used throughout the session.
  - Robot-app connection

    The user initiates a connection between the Android application and the robot via a dedicates connection button. The Controller establishes a dialogue that allows the user to input IP Address and Port of the robot, then it establishes UDP communication with the robot and verifies connectivity by sending a handshake packet. Upon successful connection, the application begins receiving odometry data, which is continuously update in the Model and used to calculate the waypoints.
  - Waypoint creation

    Waypiints are created by tapping on the gamefield map. Each tap event is captured by the Controller, converted into percentage-based coordinates relative to the map, and stored in the Model. The View then renders the waypoint as a circle button with text display it's coordinate.
  - Waypoint editing

    Users can modify existing waypoints by clicking on the Edit button on the control panel. This triggers a dialogue which lets the user inputs the ID and the new coordinate for the waypoint in meters. The Controller updates the associated waypoint data in the Model, and the View refreshed to reflect the updated location.
  - Waypoint deleting or undo-ing

Waypoints can be deleted individually (Undo) or all at once. This feature allows users to correct mistakes without restarting the entire configuration process. Deleted waypoints are removed from the Model, and the View updates accordingly.

- Waypoint sending
  Once the waypoint sequence is finalized, the user sends the waypoint data to the robot. The Send button transmit all the waypoint data on the map to the robot. Controller packages the waypoint list into binary packet and transmits them over UDP. This data includes position coordinates, execution order (ID), version (plan ID), and related parameters.
- Navigation execution
  After eceiving the waypoint data, the robot executes the navigation sequence autonomously. During execution, odometry feedback is sent back to the Android application. The Model updates the robot's position and orientation to calculate the next waypoint that can be added by the users.

- Design considerations
  The Target_Setter app UI is designed based on these criteria:
  - Usability
    The interface prioritizes touch-based interaction to reduce reliance on external controller. Tapping directly on the map allows users to define target positions.
  - Scalability and responsiveness
    The use of ConstraintLayout and percentage-based positioning ensures that the UI scales correctly across different screen sizes and resolutions. UI updates are event-driven rather than continuously refreshed, reducing unnecessary rendering operation.
  - Consistency of coordinated representation
    Storing waypoint positions as percentage-based coordinate simplifies conversion between screen space, pixel space, and real-world meter units. This design maintains consistency between the Android interface and the robot's navigation frame.
  - Separation of concerns
    By following the MVC architecture, UI rendering, user interaction handling, and data management are separated. This improves maintainability and allows individual components to be modified without affecting the entire system.
  - Limitation fo architecture
    As system complexity increased, asynchronous operations between the UI thread and network communication introduced race conditions. This limitation highlights the need for improved state management and suggests that an MVVM architecture with observable data binding would be more sustainable for future development.

- Communication protocol
  - Overview
    Communication between the Android application and the robot is implemented using a client-server communication model.

    The Android app acts as the client, responsible for generating and transmitting waypoint data, while the robot acts as a server, responsible for receiving navigation commands and executing them within ROS2 environment.

The protocol is designed to be lightweight, low-latency, and simple to integrate with existing ROS2 nodes.

- Network Model
  The communication system follows a bidirectional UDP communication. Bidirectional link enable two-way data flow with feedback mechanisms that allows the robot to send feedback back to the app. In the context of this project, the implementation is designed as:
  - App to robot: waypoint sequences, edits, and return commands are sent from the app to the robot
  - Robot to app: odometry data is transmitted at 10 Hz to provide continuous feedback.

  This bidirectional model allows the robot to navigate autonomously while keeping the app informed of the robot's real-time positions and orientation. UDP is chosen for its low-latency properties, accepting occasional packet loss because waypoint commands are repeatedly sent when necessary.

  Feedback at this frequency (10 Hz) gives smooth feedbacks, it consumes processing and network resources. On the other hand, if we eliminate the feedback, it could cause drift in calculations of the next waypoint, because the app will have to reuse the previous waypoints as the input, and since there is localization and control errors, the future waypoint calculation may not be precised. From careful obervation, 10 Hz is a balance between responsiveness and resource usage without compromising precision.
- Transport protocol selection
  In this system, we implement UDP (User Datagram Protocol) as the transport protocol. UDP is a lightweight, connectionless protocol that allows fast transmission of data without requiring a handshake or guaranteed delivery.

  This choice is sustainable for real-time robot navigation because:
  - Low latency: commands and odometry updates are delivered quickly
  - Tolerance for packet loss: occasional lost packets do not critically affect robot navigation because waypoint commands can be resent
  - Simplicity: easy to implement and integrate with ROS2 nodes.

  Unlike TCP (Transmission Control Protocol), UDP does not guarantee packet order or delivery, but for this application, the speed of communication is more critical than absolute reliability.
- Message types
  The communication between the Android app and the robot is implemented using binary UDP packets, with Little Endian byte order for efficient transmission. Binary transmission sends raw bytes instead of textm which reduces packet size and ensure faster communication – important for real-time robot control. Each packet is designed to convey specific data, such as waypoints, edits, return commands, or odometry feedback.

  Multi-byte numerical values, such as integers or doubles, are transmitted using Little Endian format. In Little Endian, the least significant byte is sent first, followed by bytes of higher significance. This ensures that data is interpreted consistently across

different devices and platforms. For example, a 32-bit integer 0x12345678 would be transmitted as 78 56 34 12.

Using Little Endian avoids misinterpretation of multi-byte values, which is critical for robot positioning and navigation accuracy.

In this system, there are multiple types of data packet, including:

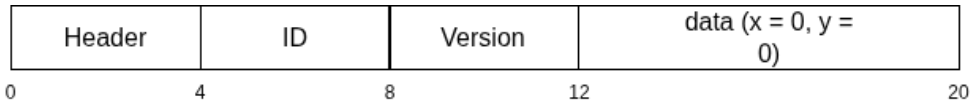
- Handshake packet: used to establish a connection between the app and the robot, confirming that both ends are ready for communication.
- Waypoint packet: sends a list off target positions for the robot to follow.
- Edit waypoint packet: updates a previously sent waypoint.
- Return message packet: sends a boolean signal for the robot to return.
- Odometry data packet: receives the robot's current position and orientation at 10 Hz.

- Packet structure

  The system defines several types of packets, each with specific role in robot navigation:

  - Handshake packet

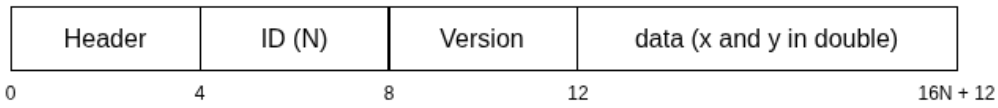
    The handshake packet is used to establish a connection between the Android app and the robot before sending actual waypoint data. When the user initiates connection, a dummy waypoint with ID -1 and zeroed coordinates is sent to the robot. This allows the robot to confirm that the communication channel is operational. The structure is shown in the diagram below.



    The robot interprets this dummy waypoint as a handshale request and responds accordingly, ensuring the client-server connection is active before transmitting real waypoint data.

  - Waypoint packet

    The waypoint packet transmits a sequence of target positions for autonomous navigation. Each waypoint contains positional coordinates, execution order, and plan ID.

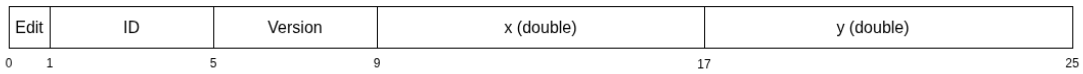


  -

    Coordinates are converted from percentage-based values in the Android UI to meters for robot navigation.

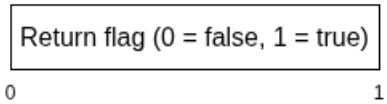
  - Edit waypoint packet

    This packet allows modification of previously sent waypoint.

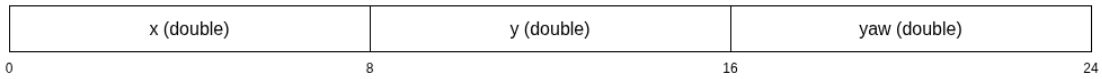


  - Return message packet

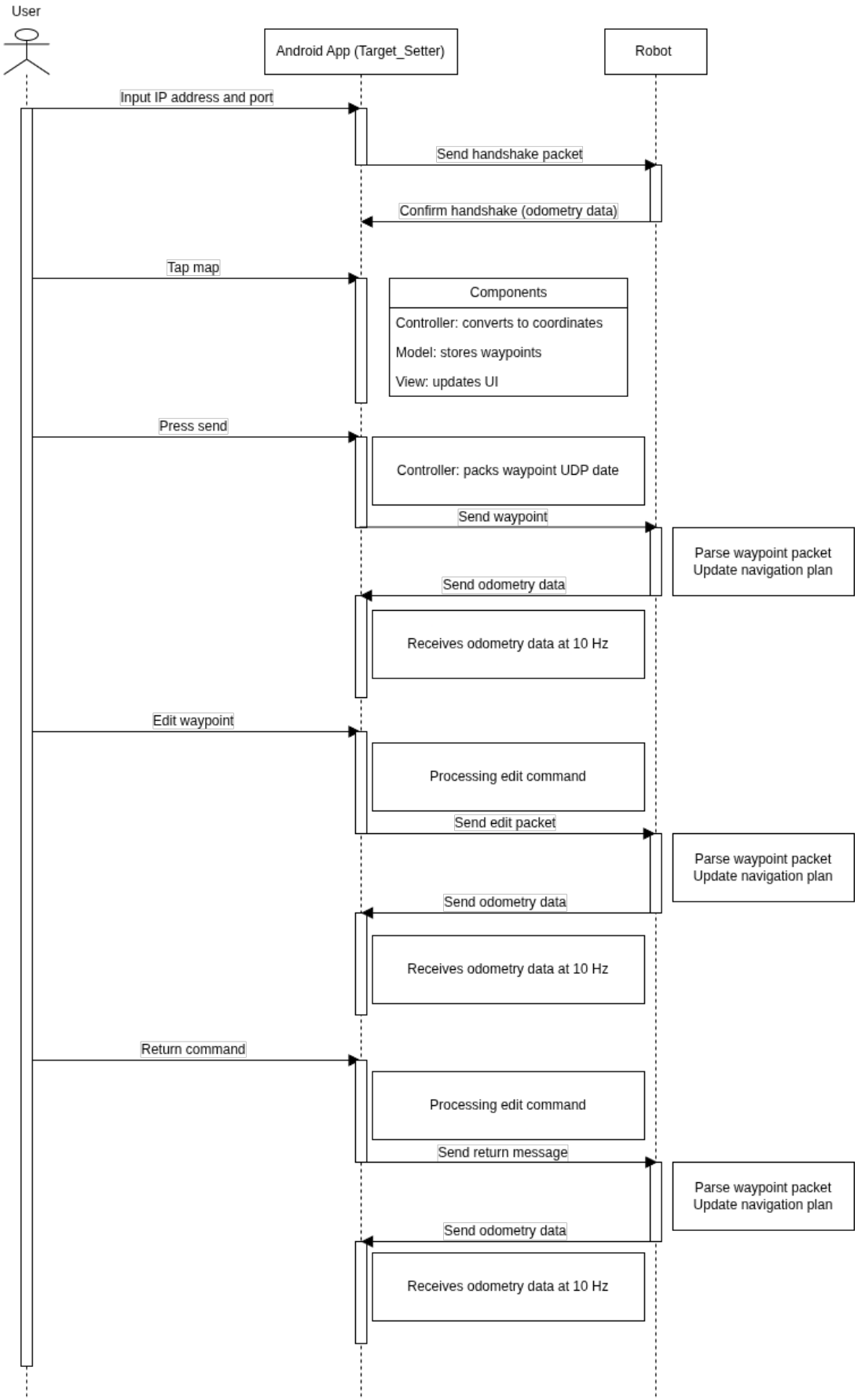
The return packet sends a command to instruct the robot to return to the previous waypoint or perform other return behaviors.



- Odometry data packet
  Odometry feedback is sent from the robot to the Android app at 10 Hz to provide continuous position and orientation updates.



This packet enables the Android app to correctly calculate the next waypoint accurately.


User
Android App (Target_Setter)
Robot
Input IP address and port
Send handshake packet
Confirm handshake (odometry data)
Tap map
Components
Controller: converts to coordinates
Model: stores waypoints
View: updates UI
Press send
Controller: packs waypoint UDP date
Send waypoint
Parse waypoint packet
Update navigation plan
Send odometry data
Receives odometry data at 10 Hz
Edit waypoint
Processing edit command
Send edit packet
Parse waypoint packet
Update navigation plan
Send odometry data
Receives odometry data at 10 Hz
Return command
Processing edit command
Send return message
Parse waypoint packet
Update navigation plan
Send odometry data
Receives odometry data at 10 Hz

- Communication flow: The operation of communication between the Android and the robot is described below.
  - Connection initialization: The user inputs the robot's IP address and port in the Android app. The app sends a handshake packet to the robot to verify connectivity. The robot responds to confirm the communication channel is active.
  - Waypoint transmission:
    The user inputs necessary parameters into the app, the tap on the screen. This created and added the waypoint to the app. The app, then, store all the data from the waypoint into a list of binary commands, that later will be sent to the robot. When the user presses Send, the existing UDP packets containing the waypoints is sent to the robot in binary format.
  - Navigation execution
    Upon receiving the waypoint data, the robot executes the navigation sequence using its ROS2 nodes.
  - Odometry feedback
    While navigating, the robot continuously transmits odometry data to the Android app at 10 Hz. This allows the app to use this data to calculate the new waypoint in case more waypoints are added later.
  - Waypoint edition/return command
    The user can edit waypoints or send a return command. These instructions are transmitted from the app to the robot, which adjusts its navigation accordingly.
  - Continuous bidirectional communication
    The app and robot maintain an ongoing exchange of data, ensuring that autonomous navigation is performed while the app keeps tracking the odometry that is sent from the robot as reference for future calculation.
- Error handling and limitations
  - Current implementation
    The Target_Setter app does not include explicit error handling mechanisms for network issues, packet loss, or odometry inconsistencies. The system relies on robot's ROS2 nodes to handle errors locally and on repeated waypoint transmissions to ensure navigation commands are received.
  - Potential issues
    - Lost or out-of-order UDP packets could result in waypoints being skipped or evacuated incorrectly
    - Connection failures may cause the app to stop receiving odometry feedback or fail to send new waypoints
    - Invalid waypoint input could result in out-of-bounds navigation if not manually checked
    - Odometry drift over long distances could misalign the robot with the expected map positions
  - Future improvements
    Implementing error handling in the app could include:
    - Checksums for packet integrity
    - Sequence numbers to detect missing or out-of-order commands
    - Validation of waypoint coordinates before sending
    - Connection retry and user notifications

These additions would increase app complexity and require careful management of asynchronous operations in the current MVC architecture. Transitioning to MVVM or using observer patterns could make handling errors more robust.

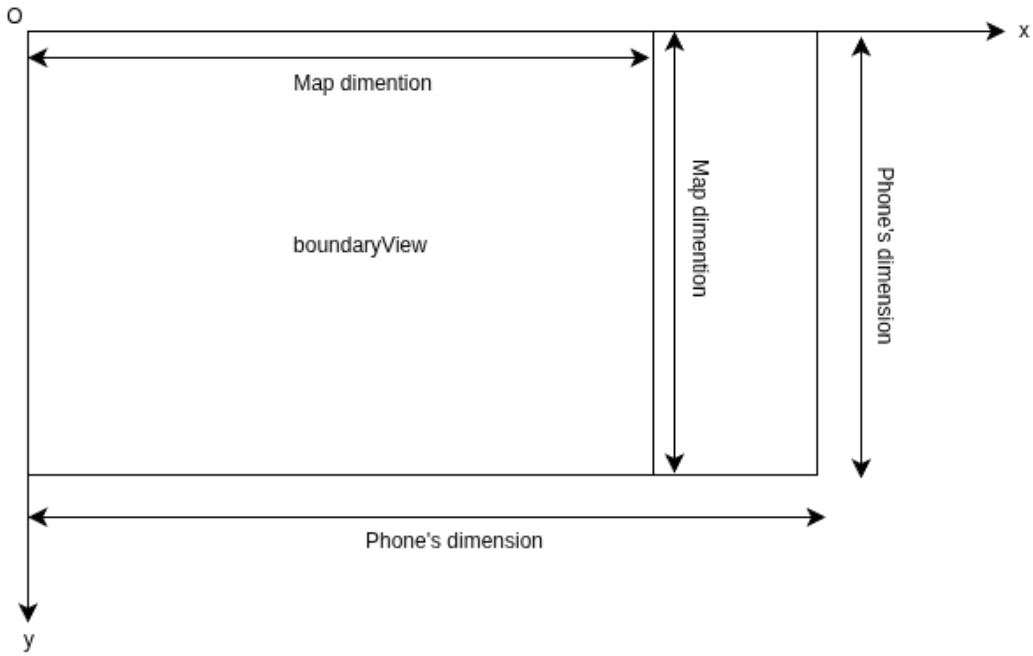
- Coordinate scaling
  - Problem definition
    In the Target_Setter Android app, waypoints are created through touch interactions on a 2 dimensional map displayed on the screen. The raw touch inputs are expressed in pixel coordinates relative to the device display.

    However, the robot operates in a global navigation frame defined in meters. Therefore, a coordinate transformation mechanism is required to convert screen-based pixel coordinates into real-world positions compatible with the robot's navigation system.

    Additionally, differences between the Android screen coordinates system and the robot's global reference frame must be considered. In Android, the origin is located at the top-left corner of the screen, with the x-axis increasing to the right and the y-axis increasing downward. In contrast, the robot's navigation frame follows a conventional Cartesian coordinate system in meters.

    To ensure consistent navigation behavior, a structured scaling and transformation process is implemented.
  - Transformation Process
    The coordinate conversion from screen touch input to global navigation coordinates consists of four stages:
    - Boundary translation
      The raw touch coordinates are first translated relative to the gamefield boundary to remove UI layout offsets.
    - Normalization
      The translates pixel values are normalized with respect to the width and height of the displayed map. This produces dimensionless values in the range [0, 1].
    - Scaling to physical dimensions\
      The normalized values are scaled according to the user-defined gamefield dimension (width and height in meters).
    - Axis adjustment
      If required, axis orientation differences between screen space and navigation frame are corrected to maintain consistency.
  - Mathematical Formulation

The map on the phone has a distinct dimension from the phone screen due to the layout called boundaryView, which was created to make it easier to add the control panel at the left pain. Thus, the touch input has offset from the phone screen



dimension.

To shift from screen coordinate to map coordinates, we apply a translation transformation:

$$x_m = x_r - x_{offset} \quad \text{and} \quad y_m = y_r - y_{offset}$$

where x_m is the waypoint x-coordinate on the map
y_m is the waypoint y-coordinate on the map
x_r is the raw touch input x-coordinate
y_r is the raw touch input y-coordinate
x_{offset} is the offset between the phone's x-dimension and the map x-dimension
y_{offset} is the offset between the phone's y-dimension and the map y-dimension

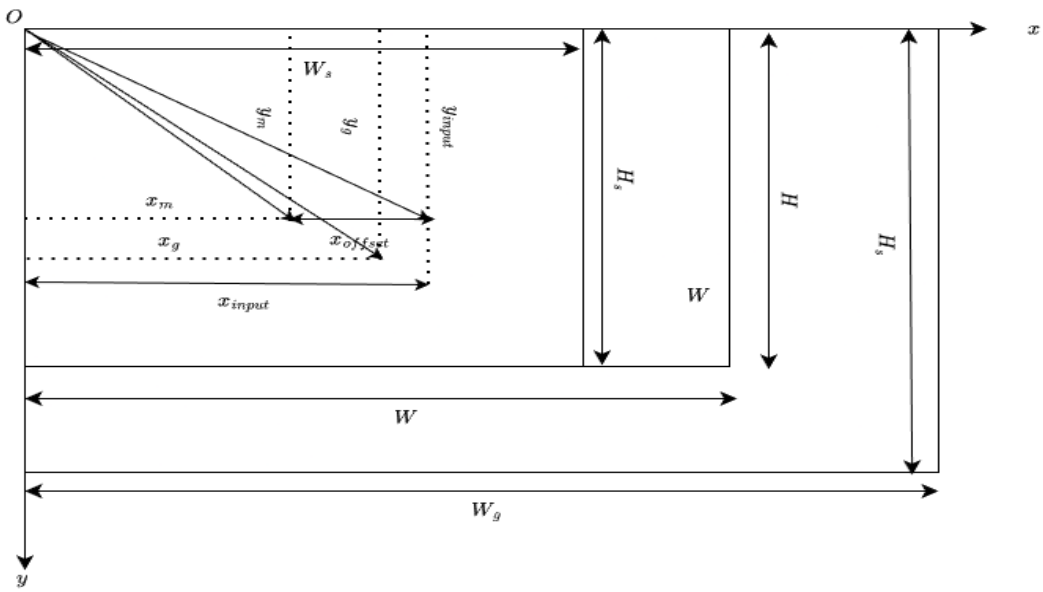
This is derived from basic coordinate translation $P_m = P_r - t$

where $p_r = (x_r, y_r)$ is raw touch input
$t = (x_{offset}, y_{offset})$ is translation vector
$p_m = (x_m, y_m)$ is map-relative coordinate

In matrix form $\begin{bmatrix} x_m \\ y_m \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_r \\ y_r \end{bmatrix} - \begin{bmatrix} x_{offset} \\ y_{offset} \end{bmatrix}$

This is rigid translation in $\mathbb{R}^2$

Since we will need to rescale into real-world Cartesian coordinate system to send the waypoint to the robot, we need the waypoint to be dimensionless.



Now the waypoint lies inside a rectangle of size $W_m \times H_m$

We want to convert this rectangle into a unit square $[0,1]\times[0,1]$

This is done by dividing by the map dimensions.

$$x_n = \frac{x_m}{W_m} \quad \text{and} \quad y_n = \frac{y_m}{H_m}$$

where x_n is the normalized x-coordinate
y_n is the normalized y-coordinate
H_m is the map height
W_m is the map width

We want:

- if $x_m = 0 \Rightarrow x_n = 0$ and $x_m = W_m \Rightarrow x_n = 1$ (1)
- if $y_m = 0 \Rightarrow y_n = 0$ and $y_m = H_m \Rightarrow y_n = 1$ (2)

Consider the equation $x_n = \alpha x_m$ (3) and $y_n = \beta y_m$ (4)

Substitute (1) and (2) into (3) and (4) respectively, then

$$\alpha W_m = 1 \Rightarrow \alpha = \frac{1}{W_m} \quad \text{and} \quad \beta H_m = 1 \Rightarrow \beta = \frac{1}{H_m}$$

This is a linear scaling transformation where $\alpha = \frac{1}{W_m}$ and $\beta = \frac{1}{H_m}$ are scaling factors for x-axis and y-axis respectively. These equations can be written in matrix form as $\begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} \alpha & 0 \\ 0 & \beta \end{bmatrix} \begin{bmatrix} x_m \\ y_m \end{bmatrix} = \begin{bmatrix} \frac{1}{W_m} & 0 \\ 0 & \frac{1}{H_m} \end{bmatrix} \begin{bmatrix} x_m \\ y_m \end{bmatrix}$

From normalized coordinate, we can derive the coordinate in percentage for better understanding and debugging. They can be derived as

$$\%x = x_n \cdot 100 \quad \text{and} \quad \%y = y_n \cdot 100$$

To send the data to the robot, we then need to convert the percentage waypoint coordinate into Cartesian coordinate in the global reference frame.

Assume that the physical field dimension is $W \times H$ (in meters), then

$$x_g = W \cdot \frac{\%x}{100} \quad \text{and} \quad y_g = H \cdot \frac{\%y}{100}$$

where W is the screen width
H is the screen height
x_g is the x-coordinate in the global reference frame
y_g is the y-coordinate in the global reference frame

As we also displays the position on the map, we need to calculate the reverse scaling coordinate back to the map coordinate.

$$x_m = x_g \cdot \frac{W}{W_s} = x_g \cdot k_x \quad \text{and} \quad y_m = y_g \cdot \frac{H}{H_s} = y_g \cdot k_y$$

where x_m is the x-coordinate on the map
y_m is the y-coordinate on the map
$k_x = \frac{W}{W_s}$ is the x-coordinate scaling factor
$k_y = \frac{H}{H_s}$ is the y-coordinate scaling factor

- Assumptions and Limitations
  The Target_Setter Android app is designed under certain assumptions and operate within defined limitations. Understanding these factors is critical for evaluating system performance and guiding future improvements.

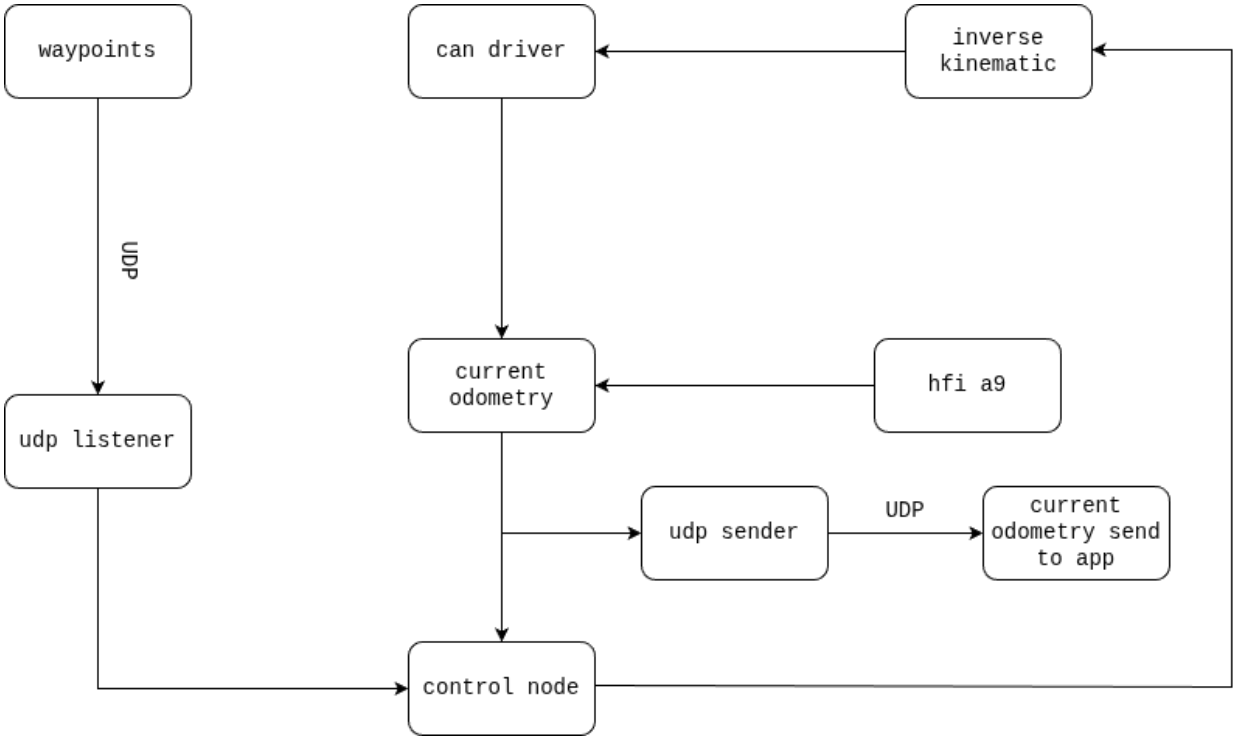
- Assumptions
  - Planar environment: the robot is assumed to operate in a two-dimensional plane. Vertical motion is neglected, and all calculations consider only x and y axes.
  - Consistent map dimensions: the displayed gamefield on the Android app is assumed to reflect the real-world field dimensions accurately. This ensures a correct mapping between screen coordinates and global coordinates.
  - Fixed screen origin and orientation: the Android coordinate system origin is located at the top-left corner of the screen, with the x-axis increasing to the right and the y-axis increasing downward. The robot's global reference frame follows a conventional Cartesian coordinate system.
  - Linear scaling suffices: conversions between screen coordinates, map coordinates, and global coordinates are assumed to be linear, using translation and scaling transformations without requiring non-linear adjustments.
  - Touch input precision: user touch input is assumed sufficiently precise to generate meaningful waypoint coordinates on the map.
- Limitations
  - Edge touch inaccuracy: touches neat the edges of the map may produce coordinates outside valid bounds if not properly clamped.
  - Screen-to-map distortion: variations in device screen resolutions and aspect ratios may introduce minor inconsistencies in scaling between the Android interface and the robot's global reference frame.
  - No rotation correction: the system assumes the map orientation and the global navigation frame are aligned. Any rotational misalignment is not corrected in the current implementation.
  - UDP communication reliability: as UDP is connectionless, packet loss or out-of-order delivery may occur, which can temporarily affect waypoint transmission or odometry feedback.
  - MVC architecture constraints: the asynchronous operations between UI rendering and network communication can introduce race conditions, potentially affecting waypoint updates and real-time accuracy.
  - Limited error handling: error detection and correction mechanisms in the Android app are minimal. The system relies on repeated waypoint transmissions and ROS2 nodes for error management, without explicit packet integrity checks or recovery strategies in the app.

- Potential Improvements

  While the current implementation of the Target_Setter app achieves basic waypoint navigation functionality, several enhancements can improve accuracy, reliability, and usability.
  - Wapoint calculation and coordinate scaling
    - Non-linear scaling adjustments: for devices with non-uniform screen resolutions or aspect ratios, introducing non-linear scaling or aspect-ratio corrections can improve mapping accuracy, especially near edges.
    - Rotation alignment: implementing a mechanism to correct rotational misalignment between the Android map and the robot's global frame would ensure waypoint accuracy even if the device or robot orientation changes.
  - Communication and networking

        - UDP packet reliability enhancements: incorporating sequence numbers, checksums, or acknowledgment mechanisms could detect and mitigate packet loss or out-of-order delivery, improving navigation reliability.
        - Adaptive feedback frequency: dynamically adjusting odometry feedback rates depending on robot speed or waypoint density could balance system responsiveness with network and processing load.
    - User interface and architecture
        - Transitioning to MVVM architecture: shifting from MVC to MVVM with observable data binding could simplify asynchronous operations, reduce race conditions, and improve the maintainability of the application as features scale.
        - Enhancing waypoint editing tools: adding visual guides, snapping to a grid, or automatic boundary clamping could improve user experience and prevent invalid waypoint input.
        - Error notifications: implementing real-time alerts for lost connections, invalid waypoints, or odometry discrepencies could improve user awareness and control during navigation sessions.
- Robot's source code

  While the Android app handles waypoint ceation and communication, the robot's source code (r2_src) is responsible for navigation for navigation execution within the ROS2 framework.



The robot software is implemented as a set of ROS2 nodes responsible for communication, control logic, state management, and odometry processing.

- Architecture and nodes

The system follows a modular ROS2 node architecture, separating the robot's responsibilities into independent functional nodes:

- udp_listener: receives UDP packets from the Android app including waypoint sequences, edits, return commands, and handshake packet. The node processes these data into messages that can be forwarded to the robot_control node to execute.
- udp_sender: listens to udp_sender node to get information about IP address and port of the app once the handshake packet is received, then reuse it as the destination to send odometry, which is published from current_odometry node, to the app.
- can_driver: handles all the messages that have to be transmitted through CAN (Controller Area Network protocol) to the low-level control logic on the devices to drive them and feedback to the higher layers of the architecture.
- hfi_a9: handles IMU (Inertial Measurement Unit) reading. This node read the quaternion that gets transmitted from the HFI A9 IMU and pre-process them before the data gets sent as messages to the current_odometry node and control_node as yaw estimate.
- inverse_kinematic: provides the wheel speed of the robot. This node listens to velocity command from the robot_control node and process them into wheel speed, then publishes them as message to the can_driver node.
- current_odometry: handle pose estimation of the robot by listening to the IMU messages and encoder feedback messages. These feedbacks get process into odometry messages that is, later, sent back to the app.
- robot_control: handles the movement of the robot. This node takes input from the udp_listener and current_odometry, then process them into velocity command thaat moves the robot and update odometry properly. This node also manages state machine, ensuring that robot is moving correctly.
- custom_messages: contains the messages that support the robot operation. This node define how those messages are structured.

The robot software consists of modular ROS2 nodes. Some nodes, such as hfi_a9 for IMU processing and acn_driver for low-level CAN communication, handle specialized hardware tasks and publish processed data for use by higher-level nodes. While I did not implement the low-level logic in these nodes, I integrated and tested their outputs with the robot_control and current_odometry nodes, ensuring seamless communication and real-time navigation execution.

Below is the project structure.

```
.
├── can_driver
│   ├── can_driver
│   │   ├── can_driver.py
│   │   └── __init__.py
│   ├── LICENSE
│   ├── package.xml
│   ├── resource
│   │   └── can_driver
│   ├── setup.cfg
│   ├── setup.py
│   └── test
│       ├── test_copyright.py
```

```
│           ├── test_flake8.py
│           └── test_pep257.py
├── current_odometry
│   ├── current_odometry
│   │   ├── current_odometry.py
│   │   └── __init__.py
│   ├── LICENSE
│   ├── package.xml
│   ├── resource
│   │   └── current_odometry
│   ├── setup.cfg
│   ├── setup.py
│   └── test
│       ├── test_copyright.py
│       ├── test_flake8.py
│       └── test_pep257.py
├── custom_messages
│   ├── CMakeLists.txt
│   ├── include
│   │   └── custom_messages
│   ├── LICENSE
│   ├── msg
│   │   ├── CanMsg.msg
│   │   ├── DigitalAndAnalogFeedback.msg
│   │   ├── DigitalAndSolenoidCommand.msg
│   │   ├── EncoderFeedback.msg
│   │   ├── MotorCommand.msg
│   │   ├── PwmCommand.msg
│   │   ├── Return.msg
│   │   ├── ServoCommand.msg
│   │   ├── TargetSetter.msg
│   │   ├── UpdateWaypoint.msg
│   │   └── Waypoint.msg
│   ├── package.xml
│   └── src
├── hfi_a9
│   ├── hfi_a9
│   │   ├── hfi_a9.py
│   │   └── __init__.py
│   ├── LICENSE
│   ├── package.xml
│   ├── resource
│   │   └── hfi_a9
│   ├── setup.cfg
│   ├── setup.py
│   └── test
│       ├── test_copyright.py
│       ├── test_flake8.py
│       └── test_pep257.py
├── inverse_kinematic
│   ├── inverse_kinematic
│   │   ├── __init__.py
│   │   └── inverse_kinematic.py
│   ├── LICENSE
│   ├── package.xml
│   ├── resource
│   │   └── inverse_kinematic
│   ├── setup.cfg
│   ├── setup.py
```

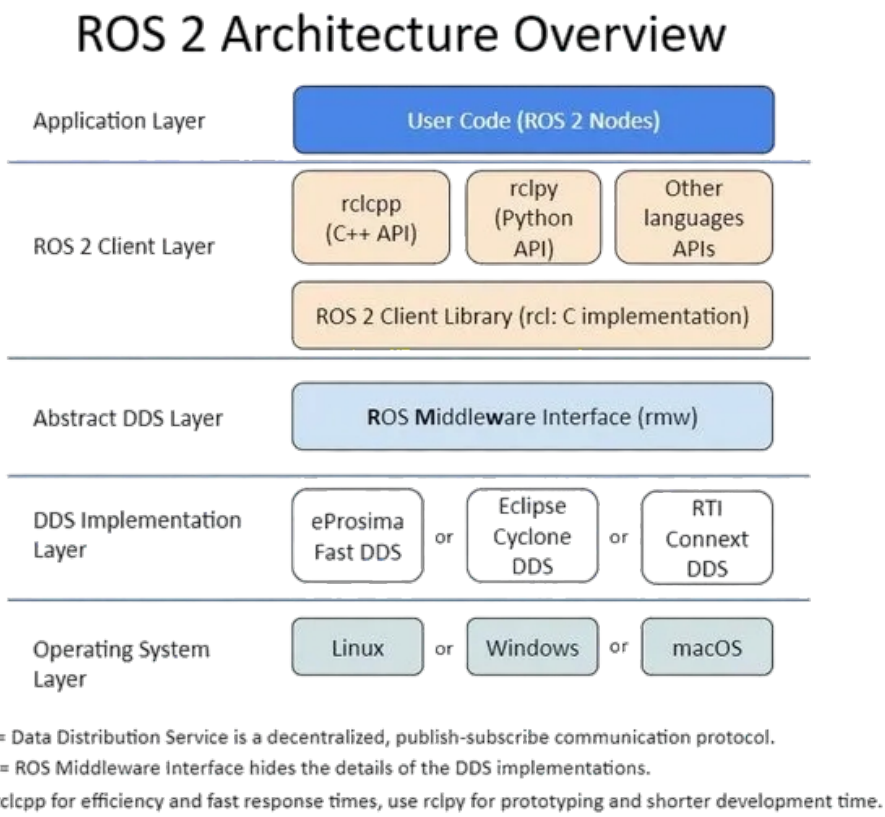
```
│   └── test
│       ├── test_copyright.py
│       ├── test_flake8.py
│       └── test_pep257.py
├── robot_control
│   ├── LICENSE
│   ├── package.xml
│   ├── resource
│   │   └── robot_control
│   ├── robot_control
│   │   ├── __init__.py
│   │   └── robot_control.py
│   ├── setup.cfg
│   ├── setup.py
│   └── test
│       ├── test_copyright.py
│       ├── test_flake8.py
│       └── test_pep257.py
├── udp_listener
│   ├── LICENSE
│   ├── package.xml
│   ├── resource
│   │   └── udp_listener
│   ├── setup.cfg
│   ├── setup.py
│   ├── test
│   │   ├── test_copyright.py
│   │   ├── test_flake8.py
│   │   └── test_pep257.py
│   └── udp_listener
│       ├── __init__.py
│       └── udp_listener.py
└── udp_sender
    ├── LICENSE
    ├── package.xml
    ├── resource
    │   └── udp_sender
    ├── setup.cfg
    ├── setup.py
    ├── test
    │   ├── test_copyright.py
    │   ├── test_flake8.py
    │   └── test_pep257.py
    └── udp_sender
        ├── __init__.py
        └── udp_sender.py

33 directories, 84 files
```

The robot software is implemented using modular ROS2 nodes, following a distributed architecture rather than monolithic design.

ROS2 architecture is built on DDS (Data Distributed Service) which is an open-standard data communications framework designed to facilitate real-time, scalable, and reliable communication between distributed system. In this real-time distributed system, each functional component of the robot runs as an independent node that

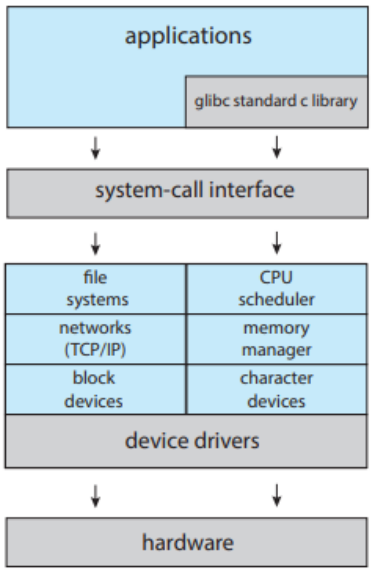
communicate asynchronously with other nodes through standardized messages. This approach supports:

- 

Concurrent operations: navigation, sensor reading, and communication run simultaneously without blocking each other
- Deterministic timing: critical for accurate odometry and precise velocity commands
- Scalability: new functionalities or nodes can be added without rewriting the entire system
- Fault isolation: errors in one node do not crash the entire system

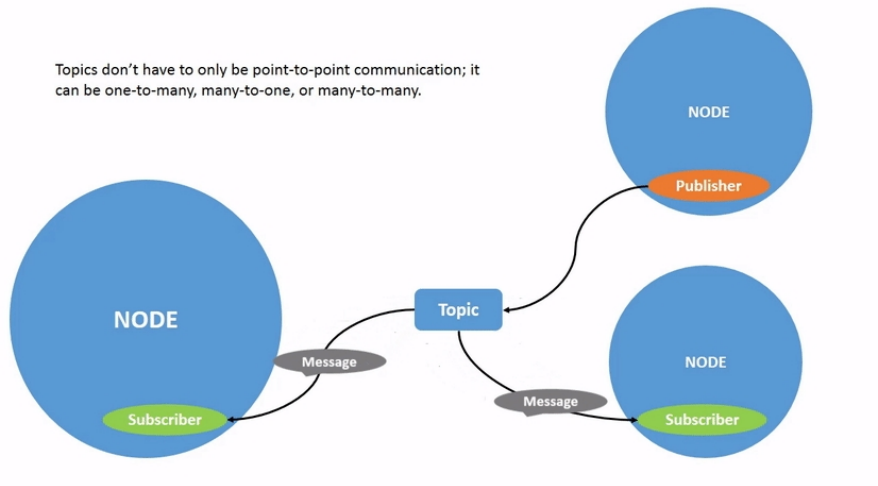
On the contrary, a monolithic architecture, where all the functionality is implemented in a single program, presents several limitations:

- Blocking operations: a delay or heavy computation in one part can freeze the entire system, affecting real-time control
- Reduced maintainability: adding or modifying functionality often requires touching unrelated modules, increasing the risk of introducing bugs
- Limited scalability: extending functionality often leads to complex dependencies and tightly coupled code
- Lower fault tolerance: failures propagate throughout the system, potentially stopping navigation or losing odometry feedback



This architectural design choice allows the Android app to communicate effectively with the robot while maintain accurate waypoint execution and odometry updates.

- Nodes communication
  In ROS2, nodes communicate using topics and messages in publish-subscribe patterns. A node is a process that performs specific functions. To operate the robot, we need multiple nodes that can communicate together. One node can send message to another node using a topic. A topic is a channel where the data is published for another node to subscribe to. In publisher-subcriber model, each node on the topic doesn't have to know who the publisher or who the subscriber is. The publisher is the node that sends the message to the subscriber, and the subscriber is the node that listens to the publisher for the message on the topic. A message is a structured data type sent over a topic, defining the contents.



When a user sends a waypoint from the Android app, the usp_listener node receives the waypoint and publishes it on the /waypoints topic. The robot_control node subscribes to this topic, calculates the required velocities, and sends commands to the inverse_kinematic node, which converts them into wheel speeds. These are finally transmitted to the robot's motors through the can_driver node. Meanwhile, the current_odometry node continously publishes the robot's position can orientation to allow real-time tracking on the Android app.

Using a distributed node architecture provides several benefits: each task runs concurrently without blocking others, errors in one node do not crash the system, new functionalities can be added easily, and real-time operation is maintained for accurate navigation.

- State machine
  The robot's waypoint navigation is controlled by a state machine implemented in the robot_control node. The state machine (or Finite State Machine: FSM) is a mathematical model of computation. Mathematically speaking, FSM is constructs the output signal one symbol at a time by observing the input signal one symbol at a time. It consists of a finite set of input symbols, a finite set of internal state, a finite set of output symbols, an initial state, a next-state function, and an output function. This abstraction allows systems to have predictable behavior in response to discrete events. In this project, the state machine defines discrete states representing the robot's

operational phases and manages transitions based on sensor feedback, user commands, or internal events.

The state machine ensures:
- Deterministic navigation behavior
- Safe handling of waypoint execution
- Clear separation between all phases
- Robust reaction to stop commands or communication loss

It prevents:
- State conflicts
- Overshooting waypoints
- Undefined behavior during rapid command updates

The system has 5 primary states:
- IDLE: the robot has no active target and waits for new waypoints or return command
- NAVIGATING: the robot is actively moving toward an assigned waypoint
- RETURNING: the robot moves back along previously visited waypoints, typically triggered by a return command
- EDIT: the robot's current waypoint is being updated in real-time, allowing for dynamic corrections
- PAUSED: the robot stops motion temporarily, either manually via a keyboard command or automatically during target reached

- State definitions
  - IDLE: the robot is stationary and waiting for waypoint data. The robot enters the IDLE state if system starts or all waypoints are completed. After completing the IDLE state, the robot transition to NAVIGATING state.
  - NAVIGATING: in this state, the robot executes motion toward the active waypoint, continuously updating odometry, calculating errors, and controlling velocity via the PD controller.
  - RETURNING: the robot navigates back along previously visited waypoints, maintaining dafe transitions similar to NAVIGATING.
  - EDIT: active waypoint edits are applied incrementally using a smoothing factor (ALPHA) to prevent sudden changes in trajectory. After editing, the robot resumes navigation.
  - PAUSED: all motion commands are set to zero velocity. The system remains paused until a resume command or automatic pause duration expires.
- State transition
  The robot's state transitions are governed by a finite state machine with 5 primary states as listed above. From IDLE, the robot transitions to NAVIGATING when a new waypoint becomes active, or to RETURNING if a return command is issued and there are visited waypoints. While navigating toward a target, reaching the waypoint triggers a transition back to IDLE. If a waypoint is updated dynamically, the robot temporarily enters the EDIT state to adjust the target before resuming navigation. The PAUSED state can be entered manually via keyboard input or automatically upon reaching a waypoint, and the robot exits PAUSED to return to either NAVIGATING or IDLE depending on whether an active target exists. These

transitions ensure smooth, predictable behavior while accommodating real-time updates and manual control.

Notes:

- Transitions always involve controller reset or motion stop to prevent abrupt movements
- Active waypoint edits and pauses are interrupt-driven, meaning they temporarily override navigation without losing track of targets
- Return navigation leverages the visited waypoint stack, ensuring a safe and predictable backward traversal

- Control logic

  The robot must move toward dynamically assigned waypoints while maintaining smooth and stable motion. Motion control must account for real-time position updates from odometry and allow safe handling of target edits or temporary pauses.

  A Porportion-Derivative (PD) controller is used to compute linear and angular velocities. The proportional term drives the robot toward the waypoint based on position error, while the derivative term provides damping based on the rate of change of the error. An integral term is omitted to prevent accumulation of outdated errors, which is especially important because the robot frequently receives new or updated waypoints.

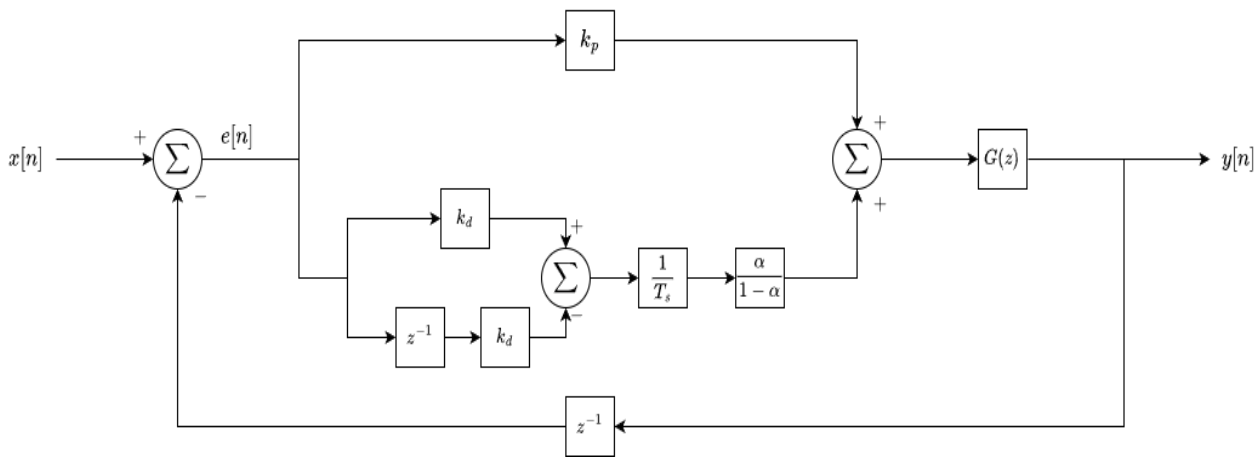
  Implementing a full PID (Proportion-Derivative-Integral) controller is not ideal for our robot for several reasons:

  - Dynamic waypoint: the robot frequently receives new waypoints or edits to existing waypoints. In such a constantly changing environment, the integral term can accumulate historical errors that are no longer relevant, leading to overshoot, oscillation, or instability.
  - Transient navigation: each waypoint represents a short-term goal rather than a persistent setpoint. PD control is sufficient to drive the robot toward the current waypoint while smoothing motion, without the need to correct long-term cumulative error.
  - Simpler and more predictable behavior: eliminating the integral term reduces computational complexity and makes the system more predictable, which is crucial for safety and smooth real-time motion control.

Let the robot's current position be (x_c, y_c, θ_c) and the desired waypoint be (x_d, y_d, θ_d)

The position errors are $e_x = x_d - x_c$ and $e_y = y_d - y_c$ or in matrix form

$$e_p = \begin{bmatrix} e_x \\ e_y \end{bmatrix} = \begin{bmatrix} x_d - x_c \\ y_d - y_c \end{bmatrix}$$



The position errors are calculated in global frame

The linear velocity command in global frame is then

$$v_g = \begin{bmatrix} v_x \\ v_y \end{bmatrix} = k_p \cdot e_p + k_d \cdot \dot{e}_{p,filtered} = \begin{bmatrix} k_p & 0 \\ 0 & k_p \end{bmatrix} \begin{bmatrix} e_x \\ e_y \end{bmatrix} + \begin{bmatrix} k_d & 0 \\ 0 & k_d \end{bmatrix} \begin{bmatrix} \dot{e}_x \\ \dot{e}_y \end{bmatrix}$$

The desired heading is $\theta_d = \arctan 2(e_y, e_x)$

The yaw error is $e_\theta = \theta_d - \theta_c$

The PD controller then computes angular velocity $\omega = k_{p,\theta} e_\theta + k_{d,\theta} \dot{e}_{\theta,filtered}$ (3)

To convert from the global frame to the local frame, we apply a 2D rotation matrix using the robot's current yaw $v_l = \begin{bmatrix} v_x^l \\ v_y^l \end{bmatrix} = \begin{bmatrix} \cos\theta_c & \sin\theta_c \\ -\sin\theta_c & \cos\theta_c \end{bmatrix} v_g$ (4)

The derivative term reacts to the rate of change of the error. However, in real system, sensor noise can cause the derivative to spike, producing jerky commands. To address this, a low-pass filter is applied to:

- Reduce noise: the filter smooths rapid fluctuations from sensor readings, preventing sudden jumps in velocity commands that could destabilize the robot
- Stabilize navigation: by damping high-frequency noise, the filtered derivative ensures the robot responds smoothly to actual motion errors rather than reacting to spurious measurements.

- Improve safety and comfort: smooth velocity commands reduce wear on the motors and provide safer, more predictable movement, especially when approaching a target or making precise maneuvers.

The derivative of the error is filtered using $\dot{e}_{p,filtered} = \alpha_d \cdot \dot{e}_p + (1-\alpha_d) \cdot \dot{e}_{p,prev}$ and for yaw error $\dot{e}_{\theta,filtered} = \alpha_d \cdot \dot{e}_\theta + (1-\alpha_d) \cdot \dot{e}_{\theta,prev}$

The PD controller computes velocity commands as $u = \begin{bmatrix} v_x^l \\ v_y^l \\ \omega \end{bmatrix}$

However, as the robot approaches the target waypoint, the velocity output is decreasing over time. As the velocity continues to decrease, it crosses a threshold where it is no longer stable. This behavior causes the robot to overcorrect itself, which results in drifts. So, to compensate for this, we add a deadzone.
Let the velocity threshold be ϵ

The deadzone can be applied as $if\, |v| < \epsilon, v = 0$

For safety reason, we need to cap the velocity once the velocity exceeds its maximum values to prevent robot from moving too fast towards the target. This can be applied as $|v| \leq v_{max}$ and $|\omega| \leq \omega_{max}$

- Sampling rate
  In a robotic control system, the sampling rate defines how often the robot reads sensor data and updates its control commands. It is the inverse of the sampling period T_s and is defined as $f_s = \frac{1}{T_s}$ where f_s is the sampling rate (Hz) and T_s is the time interval between consecutive control updates (seconds).

  In the robot's control system, the sampling rate determines how frequently:
  - Sensor data is acquired: the robot reads its current position and orientation from odometry
  - Waypoints are processed: any new target or updated waypoint is considered
  - Control commands are computed and sent: linear and angular velocities are recalculates and published.

A higher sampling rate allows the robot to react faster to changes in position, velocity, or dynamic obstacles, improving accuracy and stability. However, very high rates increases computational load and will be limited by communication latency or hardware constraints.

In our system, the control loop timer is set to 0.1s, giving sampling rate of 10 Hz. This was selected top balance responsiveness and computational efficiency. A higher sampling rate would allow the robot to react more quickly to changes in position or updates waypoint, improving precision in navigation. However, excessively high rates increase the computational load on the system, potentially causing delay in processing sensor data or publishing control commands, and may also lead to communication bottlenecks between nodes. Conversely, a lower sampling rate could reduce

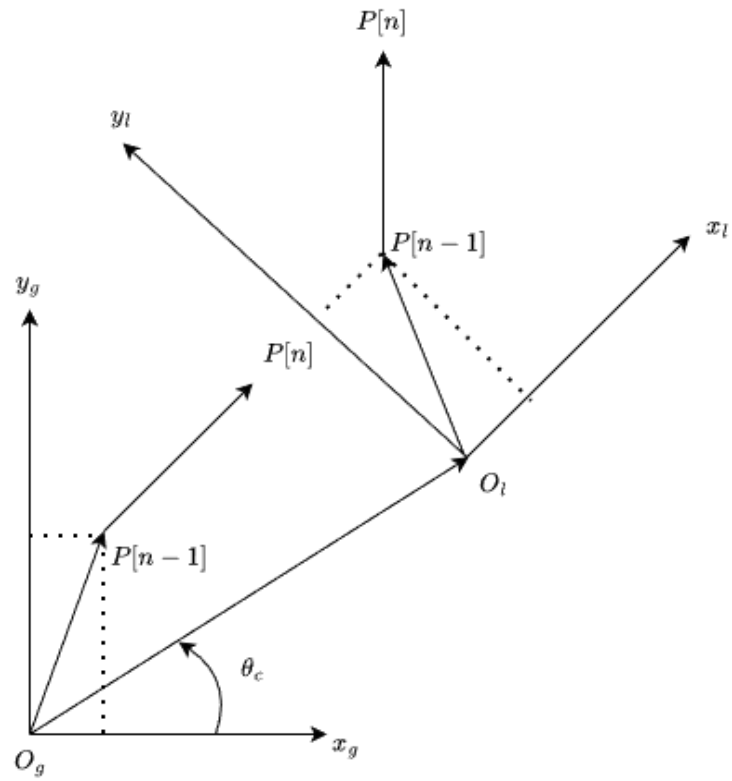
computational demands but risk delayed responses, causing oscillations or overshoot when approaching a waypoint. Based on the robot's expected speed, waypoint density, and the processing capabilities of the onboard system, 10 Hz provides sufficient responsiveness while maintaining smooth and stable navigation. This rate ensures that odometry readings and PD control updates occur frequently enough to minimize control errors without overwhelming the system.

- Tuning
  The PD controller gains were selected through iterative tuning to achieve stable and responsive navgiation. The proportional gains k_p determines how aggressively the robot corrects position and yaw errors. Higher k_p values reduce steady-state errors but can introduce overshoot or oscillations, while lower values yield smoother a=but slower responses. The derivative gains k_d act as a damping factor, mitigating oscillations and improving stability by accounting for the rate of change of the errors. In this system, a low-pass filter is applied to the derivative term to reduce the effect of sensor noise in odometry measurements, preventing the controller from reacting to small, high-frequency fluctuations. Gains were initially chosen based on expected robot dynamics and then refind through testing to ensure the robot reaches waypoints accurately without overshooting, while maintaining smooth motion along the path.
- Odometry handling
  In this system, we will be using odometry as inputs of PD controller for current position.

  Assume that the odometry reading is $(x_{odom}, y_{odom}, z_{odom})$ and the orientation as a quaternion $q=(\omega, x, y, z)$

  We convert the quaternion to yaw using $\theta=\arctan 2(2(\omega x+x y), 1-2(y^2+z^2))$

  This gives the robot's orientation in global frame



The global position at time t is updated correctly from the odometry measurement

$$P_t = \begin{bmatrix} x_{odom} \\ y_{odom} \end{bmatrix}$$

Using differential odometry, we update position incrimentally using velocities and yaw

$$P_{t+dt} = P_t + R \begin{bmatrix} v_x dt \\ v_y dt \end{bmatrix} \Rightarrow \begin{bmatrix} x_{odom} \\ y_{odom} \end{bmatrix}_{t+dt} = \begin{bmatrix} x_{odom} \\ y_{odom} \end{bmatrix}_t + \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} v_x dt \\ v_y dt \end{bmatrix}$$ where

$$R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$ is a rotation matrix in 2D

The rotation matrix converts local-frame velocities to global-frame displacements

Yaw measurements must be in the range $[-\pi, \pi]$

However, yaw measurement has an initial value from the quaternion. For consistency, we offset yaw reading $\theta_c = \theta - \theta_{offset}$ where θ_{offset} is the fist yaw reading from quaternion

For consistent control, we normalize yaw using

$$\theta_n = \arctan 2(\tan\theta_c) = \arctan 2\left(\frac{\sin\theta_c}{\cos\theta_c}\right)$$

We already mentioned that we must transform the velocity in global frame into local frame to use for velocity command in (4)

From (3), we replace θ_c by θ_n, we obtain $\begin{bmatrix} v_x^l \\ v_y^l \end{bmatrix} = \begin{bmatrix} \cos\theta_n & \sin\theta_n \\ -\sin\theta_n & \cos\theta_n \end{bmatrix} \begin{bmatrix} v_x^g \\ v_y^g \end{bmatrix}$

This is because we must ensure that current yaw is not out-of-range. For angular velocity, it can be obtained from (4)

We also mentioned state transitioning in state machine. From state transitioning condition, it follows that the robot enters PAUSED state after it reached a waypoint. In order to implement this, we need input from odometry to determine the entry point into PAUSED state.

We first determine the distance to the waypoint $d = \sqrt{(x_d - x_c)^2 + (y_d - y_c)^2}$

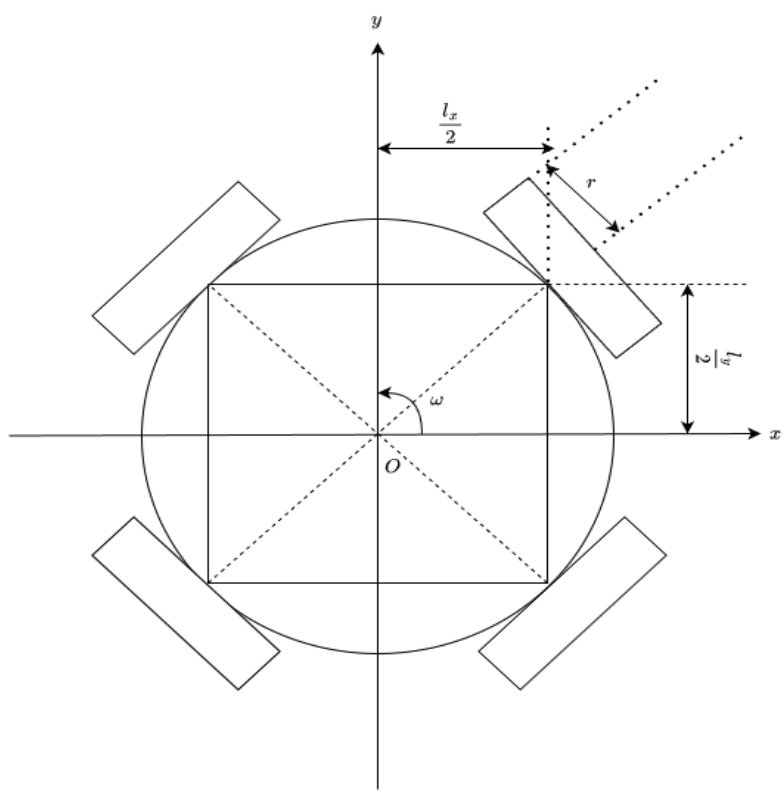
Using the distance, we can compare this to tolerance. The robot is considered to have reached the target if $d < tolerance_{pos}$ and $|e_\theta| < tolerance_\theta$

- Inverse kinematics

  Now, we need velocities as inputs for the robot to be operational.

  Assume that the robot is a rigod body. The robot is a 4-wheels omni-directional robot.

  Using the velocity of rigid body, we obtain the velocity $\vec{v_C} = \vec{v_O} + \vec{\omega} \times \vec{r_{C/O}}$



$$\vec{v_O}=\begin{bmatrix} v_x \\ v_y \end{bmatrix}, \vec{\omega}=\omega, \vec{r_{C/O}}=\begin{bmatrix} -y \\ x \end{bmatrix}$$

Then $v_C=\begin{bmatrix} v_x \\ v_y \end{bmatrix}+\omega\begin{bmatrix} -y \\ x \end{bmatrix}=\begin{bmatrix} v_x-\omega y \\ v_y+\omega x \end{bmatrix}$

For wheel i, we obtain $v_i=\begin{bmatrix} v_x-\omega y_i \\ v_y+\omega x_i \end{bmatrix}$

Suppose that r is the wheel's radius and d_i is the unit vector in wheel driving direction, then $r\,\omega_i=d_i^T v_i \Rightarrow \omega_i=\frac{1}{r}d_i^T v_i$

For front-left wheel, we have $x_1=\frac{l_x}{2}$ and $y_1=\frac{l_y}{2}$

We obtained

$$v_1=\begin{bmatrix} v_x-\omega\frac{l_x}{2} \\ v_y+\omega\frac{l_y}{2} \end{bmatrix} \Rightarrow \omega_1=\frac{1}{r}\left[\left(v_x-\omega\frac{l_x}{2}\right)-\left(v_y+\omega\frac{l_y}{2}\right)\right]=\frac{1}{r}\left[v_x-v_y-\omega\left(\frac{l_x}{2}+\frac{l_y}{2}\right)\right] \tag{5}$$

For front-right wheel, we have $x_2=-\frac{l_x}{2}$ and $y_2=\frac{l_y}{2}$

We obtained

$$v_2=\begin{bmatrix} v_x+\omega\frac{l_x}{2} \\ v_y+\omega\frac{l_y}{2} \end{bmatrix} \Rightarrow \omega_2=\frac{1}{r}\left[\left(v_x+\omega\frac{l_x}{2}\right)+\left(v_y+\omega\frac{l_y}{2}\right)\right]=\frac{1}{r}\left[v_x+v_y+\omega\left(\frac{l_x}{2}+\frac{l_y}{2}\right)\right] \tag{6}$$

For back-left wheel, we have $x_3=-\frac{l_x}{2}$ and $y_3=-\frac{l_y}{2}$

We obtained

$$v_3=\begin{bmatrix} v_x+\omega\frac{l_x}{2} \\ v_y-\omega\frac{l_y}{2} \end{bmatrix} \Rightarrow \omega_3=\frac{1}{r}\left[\left(v_x-\omega\frac{l_x}{2}\right)+\left(v_y+\omega\frac{l_y}{2}\right)\right]=\frac{1}{r}\left[v_x+v_y-\omega\left(\frac{l_x}{2}+\frac{l_y}{2}\right)\right] \tag{7}$$

For back-right wheel, we have $x_4=\frac{l_x}{2}$ and $y_4=-\frac{l_y}{2}$

We obtained

$$v_4=\begin{bmatrix} v_x+\omega\frac{l_x}{2} \\ v_y-\omega\frac{l_y}{2} \end{bmatrix} \Rightarrow \omega_4=\frac{1}{r}\left[\left(v_x+\omega\frac{l_x}{2}\right)-\left(v_y-\omega\frac{l_y}{2}\right)\right]=\frac{1}{r}\left[v_x-v_y+\omega\left(\frac{l_x}{2}+\frac{l_y}{2}\right)\right] \quad (8)$$

Let $R=\frac{l_x}{2}+\frac{l_y}{2}$ then the wheel velocity can be written in matrix form

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{bmatrix}=\frac{1}{r}\begin{bmatrix} 1 & -1 & -R \\ 1 & 1 & R \\ 1 & 1 & -R \\ 1 & -1 & R \end{bmatrix}\begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}$$

Add (5), (6), (7), and (8), we obtain $\omega_1+\omega_2+\omega_3+\omega_4=\frac{4}{r}v_x \Rightarrow v_x=\frac{r}{4}(\omega_1+\omega_2+\omega_3+\omega_4)$

Add -(5), (6), (7), and -(6), we obtain

$$-\omega_1+\omega_2+\omega_3-\omega_4=\frac{4}{r}v_y \Rightarrow v_y=\frac{r}{4}(-\omega_1+\omega_2+\omega_3-\omega_4)$$

Add -(5), (6), -(7), and (8), we obtain

$$-\omega_1+\omega_2-\omega_3+\omega_4=\frac{4}{r}\omega\left(\frac{l_x}{2}+\frac{l_y}{2}\right)=\frac{4}{r}\omega R \Rightarrow \omega=\frac{r}{4}\frac{1}{R}(-\omega_1+\omega_2-\omega_3+\omega_4)$$

It can be written in matrix form $\begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}=\frac{r}{4}\begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -\frac{1}{R} & \frac{1}{R} & -\frac{1}{R} & \frac{1}{R} \end{bmatrix}\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{bmatrix}$

- Implementation in discrete time system

Since the robot operates in a discrete-time environment with a fixed sampling period T_s, all continuous-time control equations must be converted into discrete-time form. In this system, the control loop operates at 10 Hz, corresponding to a sampling period of $T_s=0.1\,s$

Let k denote the discrete-time index

At time step k, let the robot's global position be $(x_c[k], y_c[k])$, and the waypoint be $(x_d[k], y_d[k])$

The position errors are defined as $e_x[k]=x_d[k]-x_c[k], e_y[k]=y_d[k]-y_c[k]$

In matrix form $e_p[k]=\begin{bmatrix} e_x[k] \\ e_y[k] \end{bmatrix}$

Yaw error is defined as $e_\theta[k]=\theta_d[k]-\theta_c[k]$

To ensure consistent control behavior, yaw error is normalized such that

$$e_\theta[k] = (-\pi, \pi]$$

The derivative of the error is approximated using first-order backward finite difference

$$\dot{e}_x[k] = \frac{e_x[k] - e_x[k-1]}{T_s}, \dot{e}_y[k] = \frac{e_y[k] - e_y[k-1]}{T_s}, \dot{e}_\theta[k] = \frac{e_\theta[k] - e_\theta[k-1]}{T_s}$$

This approximation enables derivative computation in a digital control system.

To mitigate high-frequency noise caused by measurements, a first-order discrete low-pass filter is applied to the derivative term $\dot{e}_f[k] = \alpha_d \dot{e}_f[k-1] + (1-\alpha_d)\dot{e}[k]$

This filtering process reduces sudden spikes in derivative estimation and ensures smoother velocity commands.

The velocity components in the global frame are computed as

$$v_x[k] = k_p e_x[k] + k_d \dot{e}_{x,f}[k], v_y[k] = k_p e_y[k] = k_d \dot{e}_{y,f}[k], \omega[k] = k_{p,\theta} e_\theta[k] + k_{d,\theta} \dot{e}_{\theta,f}[k]$$

As the robot approaches a waypoint, the velocity magnitude decreases and may enter a region where motor response becomes unstable. To prevent oscillation and drift, a deadzone is introduced $v[k] = 0, |v[k]| < v_{threshold}, v[k]\, otherwise$

This deadzone is applied to all velocity components

For safety and hardware protection, velocity commands are bounded

$$|v[k]| \le v_{max}, |\omega[k]| \le \omega_{max}$$

Since wheel actuation requires velocities in the robot's local frame, the global velocity vector is transformed using a 2D rotation matrix

$$\begin{bmatrix} v_x^l[k] \\ v_y^l[k] \end{bmatrix} = \begin{bmatrix} \cos\theta_c[k] & \sin\theta_c[k] \\ -\sin\theta_c[k] & \cos\theta_c[k] \end{bmatrix} \begin{bmatrix} v_x^g[k] \\ v_y^g[k] \end{bmatrix}$$

The robot pose is updated incrementally as

$$x_c[k+1] = x_c[k] + T_s\left(v_x^l[k]\cos\theta_c[k] - v_y^l \sin\theta_c[k]\right)$$
$$y_c[k+1] = y_c[k] + T_s\left(v_x^l[k]\sin\theta_c[k]^l_{¿}\cos\theta_c[k]\right)$$
$$\theta_c[k+1] = \theta_c[k] + T_s\,\omega[k]$$

Yaw angle is normalized after each update

The Euclidean distance to the waypoint is computed as $d[k] = \sqrt{e_x[k]^2 + e_y[k]^2}$

The waypoint is considered reaches when $d[k] < d_{threshold}$

Upon satisfaction of this condition, the state machine transitions to the PAUSED state

At each time step, the local-frame velocity vector is converted into wheel angular velocities $\begin{bmatrix} \omega_1[k] \\ \omega_2[k] \\ \omega_3[k] \\ \omega_4[k] \end{bmatrix} = \frac{1}{r} \begin{bmatrix} 1 & -1 & -R \\ 1 & 1 & R \\ 1 & 1 & -R \\ 1 & -1 & R \end{bmatrix} \begin{bmatrix} v_x[k] \\ v_y[k] \\ \omega[k] \end{bmatrix}, R = \frac{l_x}{2} + \frac{l_y}{2}$

- Security and system management
  - Safety measure

    Ensuring the safe operation of the system requires multiple layers of precaution. The following mechanisms were implemented and evaluated.
    - Velocity limitation

      Velocity commands are capped at a predefined maximum linear and angular speeds. This prevents the robot from moving too quickly toward a waypoint, which could:
      - Exceed motor capabilities and cause wheel slip
      - Produce unstable motion near target
      - Pose safety risks to nearby people or equipment

      Velocity limitation works in conjunction with PD controller gains to ensure stable convergence without overshoot or oscillation.
    - Emergency stop

      An emergency stop mechanism allows the robot to immediately halt all motion is these conditions occur:
      - Unsafe manual intervention
      - Communication failure
      - System unstable

      This mechanism overrides all current velocity commands and ensures zero motor output, providing a critical layer of safety in unpredictable situations.
    - Deadzone compensation

      A deadzone threshold is applied to small velocity commands to overcome actuator nonlinearity, such as motor deadband or friction. Without a deadzone, micro-corrections near the waypoint can cause drift and oscillation. Implementing a deadzone improves final positioning stability while minimally affecting accuracy.
    - Yaw normalization

      The robot's orientation is normalized to the range $(-\pi, \pi]$ which prevents abrupt angular jumps due to angle wrapping, which could otherwise cause sudden rotational commands and destabilize motion control.
    - Input validation

      Incoming waypoints are validated before execution during waypoints reception and processing. This ensures that:
      - Coordinates lie within allowed workspace boundaries
      - Values are numerically validated
      - Malformed messages are discarded

      Input validation prevents controller errors and unintended motion resulting from corrupt or invalid messages.
    - State machine supervision

The robot operates under a well-defined state machine. Only valid states, such as NAVIGATING, allow velocity commands. When entering PAUSED or IDLE states:

- Linear and angular velocities are explicitly set to zero
- Stale commands are ignored

This prevents transient motion due to delayed messages or communication lag.

- Communication system monitoring
  The integrity of app-robot communication is monitored to detect potential anomalies, such as delayed, repeated messages, or connection health. Monitoring ensures the robot reacts only to the most recent waypoint updates and reduces the risk of unstable behavior caused by network latency or message duplication.

- Security vulnerabilities
  Despite safety measures, several potential vulnerabilities exist in the current system.
  - Message injection risk
    Unauthorized devices could publish falsified waypoint messages and interferes with robot operation. This could cause:
    - Unintended robot motion
    - Collision risk
    - Deviation from planned paths

    Mitigation strategies include authenticated message exchange and access control.
  - Odometry integrity
    The system relies entirely on wheel odometry and IMU for localization. Corrupted or spoofed odometry data could lead to:
    - Incorrect velocity commands
    - Accumulating navigation drift
    - Misalignment between robot's actual and estimated position

    Future work could integrate sensor fusion with visual data or absolute reference frame estimation to improve robustness.
  - Denial-of-Service (DoS)
    Excessive message traffic may overload the control loop, delaying updates and degrading real-time performance. A DoS attack could originate from:
    - Malicious message flooding
    - Faulty nodes generating excessive data

    Potential mitigation includes message rate limiting, priority queues, and watchdog timer.
  - IP address and port exposure
    The robot communicates over networked interfaces that is bind from the app by IP address and port input. Exposed IP addresses or open ports may allows unauthorized access or attacks. Mitigation includes:
    - Network segmentation
    - Firewall rules
    - Encrypted channels

- Data collection
  - Data logging

During system testing, real-time monitoring of robot behavior was performed through console output and ROS topics. Persistent logging of experimental runs was not retained, meaning that numerical post-processing was not possible.

Despite this, qualitative observations were recorded, focusing on:

- Robot trajectory and alignment with assigned waypoints
- Linear and angular velocity profiles
- State transitions
- Behavior near waypoint boundaries

Although no quantitative dataset is available, repeated observations allowed assessment of system stability, responsiveness, and adherence to safety measures.

- Versioning
  - Version 0.1: initial implementation
    - Basic PD control for waypoint navigation with speed limit
    - UDP-based communication protocol with JSON array packet
    - Simple trajectory execution
    - MVC-based Android architectural patterns
    - Virtual joystick yaw correction mechanism
  - Version 0.2: architectural improvements and system updates
    - ROS2 Architecture updates: removal of target_odometry node
    - Communication protocol redefinition: transition to use UDP binary packet with Little Endian byte order, introducing different packet structure for each message types
    - Control logic improvements: added low-pass filter to derivative term, deadzone, and improved yaw correction
    - Navigation logic addition: implemented mechanism for waypoint transition, low-pass filter on edit waypoints, and arrival detection
    - Android updates: removal of virtual joystick, improved calculation, modularized gamefield definition, return and edit functionalities addition
  - Version 0.3: : system refinement and state machine addition
    - Control logic improvements: improve yaw correction, refactoring robot_control node to use state machine-based navigation mechanism
    - State machine addition: introduce state machines for better navigation control

5. Result and discussion
   - Analysis
     - Version 0.1
       - Communication system behavior: UDP-based communication using JSON arrays provided sufficient communication between robot and Android app. However, the text-based JSON structure increased packet size, which could lead to delays under high-frequency updates or higher waypoint density. Additionally, because the packet structure wasn't well-defined and no packet validation, this causes the udp_listener unable to read and process some packets.
       - Control logic behavior: PD control drove the robot toward waypoints but without derivative filtering, high-frequency noise in odometry readings caused small yaw oscillations, especially near waypoints targets. These oscillations are consistent with underdamped system behavior, where sensor noise acts as a high-frequency disturbance input.
     - Version 0.2

- Control logic performance: introduction of low-pass filtering on the derivative term significantly reduced high-frequency oscillations. Deadzone implementation mitigated micro-corrections caused by motor deadband, improving final waypoint stability.
- Localization drifts: despite improvements in control logic, odometry-based drift was still observed over longer distances or repeated maneuvers. Small linear errors accumulate over time due to actuator deadzone, which reduces oscillations near the waypoint but introduces minor incremental displacement in each control step. Although yaw is measured using the IMU, translation errors inferred from odometry to estimate waypoint progress can compound, leading to positional drift in the global frame. Specifically, estimating waypoint progress from traveled distance introduces small mismatches between actual and inferred positions, which accumulate over successive movements, resulting in larger overall drift.
- Undefined state behavior: transient asynchronous state transitions were observed, causing unintended movement. These issues are caused by state transition conflicts, emphasizing the importance of explicit state supervision to ensure safe and predictable navigation.
- Insufficiency of MVC pattern: as communication system is redefined, major updates in Android app was required. MVC pattern was sufficient before this update. As communication protocol becomes more sophisticated, MVC architecture was no longer sufficient, causing multiple race conditions and state conflicts on the app. Although, manual synchronization is sufficient for now, it is not robust in the long run.

At the time of writing, the system code has been developed up to version 0.3. Testing and performance evaluation for this version are pending.

- System evaluation

  The PD controller with derivative filtering and deadzone demonstrates critically damped behavior; trajectories converge smoothly to waypoints with minimal overshoot. Velocity capping, deadzone compensation, and state machine supervision is needed to prevent unsafe acceleration or drift. Odometry drift, deadzone, incremental errors, and distance-based waypoint inference leads to micro-correction in yaw, causing oscillation as the robot navigate over longer distance. At the rate of 10 Hz control loop, the system adapts to waypoint edits dynamically with minimal observable lag. MVC architectural pattern for the Android app introduces race conditions and state conflicts as the system becomes more sophisticated with more asynchronous operations required.
- Improvement suggestions
  - Persistent data logging: implement structured logging of position, orientation, and velocity to enable quantitative analysis of trajectory error, settling time, and velocity smoothness. This would allow RMS error calculation and systematic tuning of controller gains.
  - Sensor fusion for localization: integrate IMU, visual odometry, or LiDAR with wheel odometry to reduce cumulative drift, particularly over longer distances or repeated rotations. Sensor fusion would compensate for incremental errors caused by deadzone and distance-based waypoint inference.
  - Adaptive PD gains: dynamically adjusting proportional and derivative gains based on speed, load, or trajectory curvature could reduce micro-corrections and improve overall stability, especially over extended runs.

- Enhanced state management in Android app: refactor the Android MVC architecture to minimize race conditions and state conflicts. Consider event-driven or observer patterns to synchronize asynchronous operations reliably, particularly during waypoint updates and user interactions.

6. Conclusion

The discrete-time PD controller with derivative filtering and deadzone compensation demonstrates stable and critically damped behavior for a four-wheel omnidirectional robot. Trajectories converge smoothly to dynamically assigned waypoints, and velocity limits combined with state machine supervision prevent unsafe acceleration or drift.

Although yaw is measured using an IMU, incremental errors from deadzone and distance-based waypoint inference introduce micro-corrections that accumulate over longer paths, producing minor oscillations. At a 10 Hz control loop, the system adapts efficiently to updated waypoints with minimal lag.

The iterative development and refinement of the control logic, state machine integration, and Android interface improvements provide a robust foundation for indoor navigation. Future work should focus on persistent logging, sensor fusion, adaptive control gains, and architectural refinements to further reduce drift, improve stability, and support quantitative performance evaluation.

Reference:

- S. Singh, “MVC Framework: A Modern Web Application Development Approach and Working,” 2020 International Research Journal of Engineering and Technology (IRJET), vol. 07, no. 01, pp. 51-55, Jan 2020. p-ISSN: 2395-0072, e-ISSN: 2395-0056. Available: https://www.irjet.net/archives/V7/i1/IRJET-V7I111.pdf
- A. Nandwanya, “ConstraintLayout, Demistified – how it really works and when you should reach for it, ” Android Engineers, Sep 02, 2025 . [Online]. Avaiblable: https://androidengineers.substack.com/p/constraintlayout-demystified-how [Accessed: Feb. 12, 2026]
- “MVC Framework – Architecture, ” tutorialspoint, n.a., [Online]. Avaiblable: https://www.tutorialspoint.com/mvc_framework/mvc_framework_architecture.htm [Accessed: Feb. 13, 2026]
- “Model-View-ViewModel (MVVM), ” Microsoft, n.a., [Online]. Available: https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm [Accessed: Feb. 13, 2026]
- “Build a responsive UI with ConstraintLayout, ” Android Developer, n.a., [Online] Available: https://developer.android.com/develop/ui/views/layout/constraint-layout [Accessed: Feb. 13, 2026]
- “Unidirectional Link vs Bidirectional Link vs Loopback Link, ” CCNA classes, CCNA Study Resources, Editorial, Networking Basics, May 27, 2021, [Online] Available: https://ccna-classes.com/ccna-study-resources/unidirectional-link-vs-bidirectional-link-vs-loopback-link/ [Accessed: Fed. 13, 2026]
- Cisco Network Academy, *CCNAv7: Introduction to Networks Companion Guide*. Indianapolis, IN, USA: Cisco Press, 2020.
- P. L. Dordal, "16 UDP Transport," An Introduction to Computer Networks, desktop edition 2.0.11, 2012. [Online]. Available: https://intronetworks.cs.luc.edu/current/html/udp.html [Accessed: Feb. 14, 2026].
- J. Verschelde, “Client/Server Interactions, ” Programming Tools and File Management 0.7.5 documetation, Part VI: Network Programming, 2017, [Online] Available: https://homepages.math.uic.edu/~jan/mcs275/mcs275notes/lec27.html [Accessed: Feb. 14, 2026]
- S. Narayana, “CS 352 User Datagram Protocol, CS 352, Lecture 8.1, ” Rutgers university | New Brunswick, n.a., [Online]. Available: https://people.cs.rutgers.edu/~sn624/352-S21/lectures/transport-udp-error-detection.pdf [Accessed: Feb. 14, 2026]
- “Robotic System, Section II. MODELING, Chapter 3. Coordinates and Transformations, ” n.a., n.a., [Online] Available: https://motion.cs.illinois.edu/RoboticSystems/CoordinateTransformations.html [Accessed: Feb. 15, 2026]
- Huseyin Kutluca, “Robot Operating System 2 (ROS 2) Architecture, ” Medium, Dec 6, 2020, [Online] Available: https://medium.com/software-architecture-foundations/robot-operating-system-2-ros-2-architecture-731ef1867776 [Accessed: Feb. 15, 2026]
- rti community, “Data Distribution Service (DDS) Community RTI Connext Users, ROS 2: What is DDS, ” rti community, n.a., [Online] Available: https://community.rti.com/page/ros-2-what-dds [Accessed: Feb. 15, 2026]
- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concept, Tenth Edition*, Wiley, 2018.
- Open Robotics, “Nodes, ” Ros 2 Documentation: Humble, 2026, [Online} Available: https://docs.ros.org/en/humble/Concepts/Basic/About-Nodes.html [Accessed: Feb. 15, 2026]
- Open Robotics, “Topics, ” ROS 2 Documentation: Humble, 2026, [Online] Available: https://docs.ros.org/en/humble/Concepts/Basic/About-Topics.html [Accessed: Feb. 15, 2026]

- Open Robotics, “Interfaces, ” ROS 2 Documentation: Humble, 2026, [Online] Available: http://docs.ros.org/en/humble/Concepts/Basic/About-Interfaces.html [Accessed: Feb. 15, 2026]
- S. Lipschutz, M. L. Lipson, *Schaum’s Outline Of Theory and Problem of Discrete Mathematics Third Edition*, The McGraw-Hill Companies, Inc, USA, 2007, DOI: 10.1035/0071470387
- E. Lee, P. Varaiya, *Structured and Interpretation of Signal and Systems*, Pearson Education, n.a., 2003, [Onlin} Available: https://ptolemy.berkeley.edu/projects/chess/eecs124/reading/LeeAndVaraiya3_4.pdf ISBN: 0-201-74551-8 [Accessed: Feb. 15, 2026]
- K. J. Astrom, “Control System Design, ” n.a., n.a., 2002, [Online] Available: https://www.cds.caltech.edu/~murray/courses/cds101/fa02/caltech/astrom-ch6.pdf [Accesed: Feb. 16, 2026]
- A. Boghossian, J. Brown, S. Zak, “P, I, D, PI, PD, and PID control, ” LibreTexts Engineering, University of Michigan, n.a., [Online] Available: https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Chemical_Process_Dynamics_and_Controls_(Woolf)/09:_Proportional-Integral-Derivative_(PID)_Control/9.02:_P_I_D_PI_PD_and_PID_control [Accessed: Feb. 16, 2026]
- H. P. Oliveira, et al., “Dynamic Models for Omni-directional Robots with 3 and 4 Wheels, ” ICINCO (International Conference on Informatics in Control, Automation and Robotics), 2008
- A. Phunopas, S. Inoue, “Motion Improvement of Four-Wheeled Omnidirectional Mobile Robots for Indoor Terrain, ” Atlantis Press, Journal of Robotics, Networking and Artificial Life, Vol. 4, No. 4, March 2018, 275-282
- D. C. Karnopp, D. L. Margolis, R. C. Rosenberg, *System Dynamics, Modeling, Simulation, and Control of Mechatronic Systems*, John Wiley & Sons, Inc, USA, 2012, ISBN 978-0-470-88908-4
- B.P. Lathi, *Principles of Linear Systems and Signals, Second Edition, International Version*, Oxford University Press, New Delhi, India, 2009, ISBN-13: 978-0-19-806227-1
- S. Simrock, “Control theory, ” DESY, hanburg, Germany, n.a., [Online] Available: https://cds.cern.ch/record/1100534/files/p73.pdf [Accessed: Feb. 16, 2026]